

A multi-objective task offloading and resource allocation in blockchain-enabled IoT networks

Nasim Nezhadsistani ^{a,b}, Naghmeh S. Moayedian ^a, Arash Bozorgchenani ^c,^{*}, Burkhard Stiller ^b

^a Department of Electrical and Computer Engineering, Isfahan University of Technology, Isfahan 84156-83111, Iran

^b Department of Informatics, University of Zurich, Switzerland

^c School of Computer Science, University of Leeds, Leeds, UK

ARTICLE INFO

Keywords:

Internet of Things (IoT)
Multi-objective optimization
Blockchain
Mobile Edge Computing (MEC)
Computational offloading
NSGA-II
Resource allocation
Energy efficiency

ABSTRACT

The integration of Mobile Edge Computing (MEC) and Blockchain has emerged as an efficient approach to support the development of secure and delay-sensitive Internet of Things (IoT) applications. Despite these advantages, blockchain-enabled IoT systems face a fundamental challenge because the security and trust benefits of blockchain are accompanied by additional computation and communication overheads that can degrade latency and energy efficiency in resource-constrained environments. This paper develops a unified multi-objective optimization framework for task offloading and resource allocation in blockchain-enabled IoT networks. The considered system architecture incorporates IoT devices, MEC servers, and a cloud data center while explicitly modeling blockchain operations as part of the offloading process.

A multi-objective optimization problem is formulated to jointly minimize total system latency and energy consumption while maximizing the aggregate reputation of the selected blockchain management nodes. To solve this problem, two evolutionary solution approaches are employed. The Non-dominated Sorting Genetic Algorithm II (NSGA-II) is used to identify Pareto-optimal trade-off solutions, and a Genetic Algorithm (GA) is adopted to solve a weighted-sum scalarized formulation. Comprehensive simulation results indicate a demonstrable trade-off between latency and energy consumption. Specifically, the analysis reveals that prioritizing high-reputation nodes for blockchain consensus roles significantly reduces transaction verification latency, thereby enhancing the overall system performance despite the operational overheads. The framework allows stakeholders to select solutions that best meet specific IoT application requirements.

1. Introduction

The Internet of Things (IoT) has emerged as an important paradigm in the development of the foundations of the digital infrastructure of the current era, which has enabled the interconnection of a wide range of sensing, computing, and communication technologies in various domains such as smart cities, industrial automation, intelligent transportation, healthcare, and environmental monitoring [1,2]. This has generated enormous volumes of data, placing significant pressure on data processing, storage, and management infrastructure [3,4]. Moreover, the IoT devices are generally limited in their computing, storage, and battery capabilities, thereby restricting their ability to process computationally intensive tasks locally [5,6]. Thus, efficient computation offloading has become an important requirement for satisfying the latency, reliability, and scalability demands of modern IoT applications.

Mobile Edge Computing (MEC) has been recognized as an effective architectural solution to these limitations by providing computation and storage resources in close proximity to end devices [7]. By transferring task execution from resource-constrained devices to nearby edge servers, MEC can reduce end-to-end latency, relieve backbone network congestion, and improve Quality of Service (QoS) for delay-sensitive applications [8,9]. However, the migration from centralized cloud-centric processing to distributed edge environments also introduces important system-level challenges, particularly with respect to trust management, data integrity, privacy preservation, and coordination across multiple administrative entities [10,11].

Blockchain has gained significant attention as a viable approach to addressing these issues within decentralized IoT ecosystems. Beyond its original association with cryptocurrency systems, blockchain provides a distributed and tamper-resistant ledger that enables transparent record

* Correspondence to: School of Computer Science, University of Leeds, Sir William Henry Bragg Building, Woodhouse Lane, Leeds, LS2 9JT, UK.

E-mail address: a.bozorgchenani@leeds.ac.uk (A. Bozorgchenani).

<https://doi.org/10.1016/j.adhoc.2026.104361>

Received 28 March 2026; Received in revised form 8 July 2026; Accepted 15 July 2026

Available online 25 July 2026

1570-8705/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

keeping, auditable interactions, and decentralized coordination without reliance on a fully trusted central authority [12–15]. Within IoT ecosystems, these characteristics are particularly valuable for secure data provenance, access control, and trustworthy service interactions among heterogeneous devices and stakeholders [16–18]. However, the incorporation of blockchain into resource-constrained IoT-MEC systems introduces non-negligible costs. Consensus execution, transaction dissemination, block generation, block verification, and feedback propagation may impose considerable latency and energy overheads, which can significantly degrade system performance in delay-sensitive and energy-constrained applications, particularly when computationally intensive consensus mechanisms are employed [19,20].

Although the literature on MEC-based task offloading and the literature on blockchain-enabled IoT security have both developed substantially, their intersection remains insufficiently investigated from an integrated system optimization perspective. In many existing studies, offloading and resource allocation are optimized while security-related processing costs are simplified or ignored. On the other hand, research focused on blockchain applications tends to emphasize the security benefits of blockchain architecture without explicitly modeling detailed operational costs in performance-concerned optimization models. Such a distinction is not suitable in a secure IoT-MEC environment, where computation offloading and blockchain operations compete for limited communication and computational resources and jointly affect latency and energy performance. An integrated optimization framework is thus required to consider computation offloading decisions and detailed latency and energy costs of blockchain operations.

To address this research gap, this article proposes a unified multi-objective optimization model for secure task offloading in blockchain-enabled IoT-MEC systems. In particular, the proposed optimization model accounts for both computational tasks and blockchain operations and explicitly considers trade-offs among latency, energy consumption, and blockchain management quality through node reputation. In terms of solution approaches, both Pareto evolutionary and scalar optimization are employed to tackle the proposed optimization model. The main novelty of this work should be interpreted at the system-modeling and optimization-framework level. In particular, this paper does not claim that NSGA-II or GA are newly invented algorithms. Instead, these algorithms are used as solution engines for the proposed mixed-integer nonlinear optimization problem. The primary contribution is a unified blockchain-aware IoT-MEC optimization framework that explicitly couples task offloading, computational resource allocation, blockchain-stage latency/energy overheads, and reputation-aware consensus-role assignment. The main contributions of this work are summarized as follows:

- An integrated architecture combining IoT, MEC, and cloud services has been developed, which incorporates blockchain mechanisms. In this setup, MEC/cloud nodes function as both computational servers and blockchain management nodes.
- A comprehensive multi-objective optimization problem has been formulated to minimize system latency and total energy consumption while maximizing the overall reputation of the selected blockchain management nodes. This formulation explicitly links computation offloading, CPU resource allocation, and the assignment of blockchain consensus roles.
- Blockchain operational overhead is modeled in a stage-wise manner, including block generation, block transmission, verification, broadcasting, and feedback propagation. This avoids the oversimplified assumption of a fixed blockchain delay or a single aggregated blockchain cost.
- A constraint-aware evolutionary solution framework has been developed. NSGA-II is employed to approximate the Pareto-optimal trade-off surface, while a Genetic Algorithm (GA) is utilized to identify a scalarized operating point based on the preference weights specified by the network operator. To enhance the

practicality of the solution process, techniques such as feasibility-preserving initialization, constraint-repair, and penalty-based handling are integrated.

- Extensive performance evaluation is provided, including comparisons with representative baselines, ablation studies, sensitivity analysis, dynamic channel conditions, verifier-node failures, computational-complexity analysis, and implementation-overhead discussion.

Each component of the proposed framework serves a distinct role. The MEC offloading process is designed to decrease the time needed for completing tasks through transferring computations from constrained IoT devices to adjacent edge servers. Blockchain technology offers decentralization, trustworthiness, data immutability, and auditing for the results of performed tasks and their associated services. Reputation-aware selection helps obtain better consensus node selections since it favors more reliable and competent nodes. NSGA-II can find Pareto-optimal solutions, while GA-based scalarized formulation gives an optimal solution point under given criteria.

The remainder of this paper is organized as follows. Section 2 reviews the most relevant literature. Section 3 presents the system model. Section 4 formulates the optimization problem. Section 5 describes the proposed solution methods. Section 6 reports and discusses the simulation results. Finally, Section 7 concludes the paper and outlines future research directions.

2. Related work

The challenges of resource allocation and task offloading in IoT and MEC environments have been extensively investigated, with a parallel stream of research exploring the integration of blockchain for enhanced security:

2.1. Task offloading and resource allocation in MEC

Early studies on task offloading predominantly concentrated on single-objective optimization, such as minimizing latency [29] or energy consumption [30]. These problems are typically formulated as Mixed-Integer Non-Linear Programming (MINLPs) and are known as NP-hard. As a result, heuristic and meta-heuristic algorithms have gained extensive adoption. Numerous studies have utilized Genetic Algorithms (GA) [31], Particle Swarm Optimization (PSO) [32], and simulated annealing to identify near-optimal solutions. Recently, Reinforcement Learning (RL) and Deep Reinforcement Learning (DRL) have acquired prominence for their capacity to execute dynamic offloading judgments in fluctuating contexts [33,34]. However, single-objective RL methods frequently encounter difficulties when attempting to concurrently balance several conflicting objectives. To address this, multi-objective approaches have been proposed to jointly optimize latency and energy consumption, for instance, by utilizing NSGA-II to find Pareto-optimal fronts [29]. Nevertheless, such existing multi-objective frameworks typically do not account for the additional computation and communication overheads introduced by security and privacy mechanisms like blockchain.

Coordinated resource allocation, task scheduling and intelligent offloading are also heavily highlighted in recent studies in heterogeneous edge systems. In CPSS, SerFlow explores the end-edge-cloud collaboration workflow applications and emphasizes the necessity of a service-enhanced scheduling and resource coordination in resource-limited collaborative systems [35]. There have also been attempts on the joint offloading decisions and computation resource allocation for vehicular-fog computing systems, where both computation and communication resources are jointly optimized for vehicular services with delay sensitivity [36]. Further, the optimization for digital-twin-aided vehicular networks by using the SAC-based offloading and resource-allocation optimization is found to be an important research field for

Table 1

Comparison between the proposed framework and representative related studies.

Study	MEC offloading	Resource allocation	Blockchain overhead	Reputation-aware consensus	Multi-objective	Dynamic evaluation
MEC offloading studies [21,22]	✓	✓	–	–	Partial	Partial
Blockchain-enabled MEC studies [23,24]	✓	Partial	Aggregated/fixed	Partial	–	Partial
Multi-objective offloading studies [25,26]	✓	✓	–	–	✓	Partial
DRL-based secure offloading [27]	✓	✓	Partial	Partial	Scalarized	✓
MOEA/D-based offloading [28]	✓	✓	–	–	✓	Partial
Proposed framework	✓	✓	Stage-wise	✓	✓	✓

dynamic vehicular edge systems [37]. The reliability-aware computation offloading for delay-sensitive MEC-enabled aerial computing also shows that reliability must be taken into account in addition to delay and energy goals in MEC scenarios [38].

2.2. Blockchain integration in IoT and MEC

Blockchain has been employed in IoT systems for secure data sharing [39], access control [40], and supply chain management [41]. Several studies have proposed integrating blockchain with MEC to establish a secure and decentralized edge-cloud computing continuum [42,43]. For instance [44] proposed a blockchain-based data sharing framework for MEC, and the authors in [45] designed a lightweight blockchain mechanism for industrial IoT. However, many of these studies focus mainly on security architectures while giving limited attention to the performance implications of blockchain integration (e.g., the latency and energy overheads incurred by block generation, consensus, and propagation). This oversight may lead to underestimated costs when the blockchain is deployed in latency and energy-sensitive systems [46].

2.3. Multi-objective optimization for joint problems

The authors in [47] proposed an NSGA-II-based approach for task scheduling in a UAV-enabled MEC system, deriving a set of trade-off solutions under UAV-assisted edge constraints rather than a single operating point. [6] employed NSGA-II for task offloading and resource scheduling in mobile edge-cloud computing, leveraging task prediction and an edge-competition mechanism to better capture the energy-latency trade-off in dynamic environments. [48] presented a blockchain-based resource allocation mechanism that explicitly aims at balancing efficiency and security, illustrating how blockchain considerations can reshape allocation priorities in vehicular networks.

Despite these advances, comparatively fewer studies provide a unified optimization model that simultaneously captures computation offloading decisions and detailed blockchain processing overheads in IoT-MEC systems [30,49,50]. In many existing formulations, the latency and energy implications of Blockchain Offloading Processes (BOPs) are either omitted or represented at a level of abstraction that limits the realism of system-level evaluation. Moreover, prior studies often assess security-enhanced offloading frameworks under simplified assumptions on blockchain processing dynamics, such as fixed consensus delay or aggregated communication cost, which can obscure the actual trade-offs in heterogeneous environments where blockchain functions and application tasks compete for limited resources. Consequently, the feasibility and efficiency of secure offloading strategies may be overestimated when detailed blockchain-stage costs are not explicitly modeled. In contrast, the present study develops a unified multi-objective formulation that jointly models computation offloading and blockchain-related latency-energy overheads, while also incorporating the reputation of blockchain management nodes into the optimization process.

As shown in Table 1, most research work focuses on offloading computation tasks and allocation of system resources in an MEC environment where minimizing latency and energy consumption can be considered scalarized objectives or partial multiple-objectives [21,

22]. The use of blockchain to enhance MEC improves data reliability, trustworthiness, or transaction management; however, blockchain overhead is indirectly addressed in terms of blockchain system-level metrics or through blockchain architecture design for enhanced security, rather than stage-by-stage latency-energy cost calculation [23,24]. Offloading research that considers multi-objective MEC environments can optimize delay, energy cost, or other performance metric using Pareto optimization approach; still, they do not consider blockchain consensus cost or reputation-driven node participation in blockchain consensus procedure [25,26]. Likewise, approaches based on DRL and MOEA/D offer more robustness in dynamic network environments through their respective algorithms but ignore blockchain overhead costs and reputation-driven node selection for the blockchain layer [27, 28].

3. System model

A hierarchical, blockchain-based IoT architecture for computational offloading is considered, as seen in Fig. 1. The system consists of four main entities: IoT devices, MEC servers, a central cloud server, and a decentralized blockchain network. IoT devices including sensors, actuators, and other intelligent nodes, collect data and execute computation-intensive operations. According to their limited local resources, these tasks are offloaded to nearby MEC servers. The MEC servers provide computational capacity and operate as nodes inside the blockchain network. A central cloud server, symbolizing a primary data center, provides almost limitless processing capacity and concurrently engages in the blockchain network. The blockchain provides the foundation of the decentralized trust framework, offering a secure, unchangeable, and auditable public ledger for all essential transactions (e.g., task outcomes, data access records). The network comprises MEC servers and a cloud server, functioning as blockchain nodes. Within the overall framework of this architecture, the servers assume two key roles: one as computational platforms for executing tasks that were offloaded, and another as nodes of the blockchain that are engaged in validating transactions and executing consensus. This level of functional integration is critical in ensuring that there is efficiency in the system, but at the same time raises more stringent demands in terms of resource management and coordination [51]. Once the tasks have been executed, transactions are created and propagated to the blockchain in order to log the outcomes of tasks or metadata associated with tasks. A block manager is responsible for aggregating a set of transactions into a candidate block that is appended to the blockchain after consensus among verifier nodes.

The task allocation and the distribution of resources are expressed by the binary task assignment matrix $\mathbf{X} \in \{0, 1\}^{(N+2) \times (M+1)}$, and the non-negative resource allocation matrix $\mathbf{F} \in \mathbb{R}_{\geq 0}^{(N+2) \times (M+1)}$, as depicted by (1)–(2), respectively. The columns $1:M$ denote the MEC servers, whereas the $m = M+1$ represents the cloud. The rows $1:N$ correspond to the device tasks, and the last two rows ($N+1$ and $N+2$) correspond to selecting the block manager and the verifier(s) of the blockchain network, respectively. The entry $x_{n,m} = 1$ implies that the task or the role of the blockchain in row n is assigned to the MEC server m . The entries $f_{n,m}$ denote the CPU frequency, in cycles per second, allocated to the task or role of the blockchain network by the MEC server m . It is assumed that each task is assigned to only one column of matrix

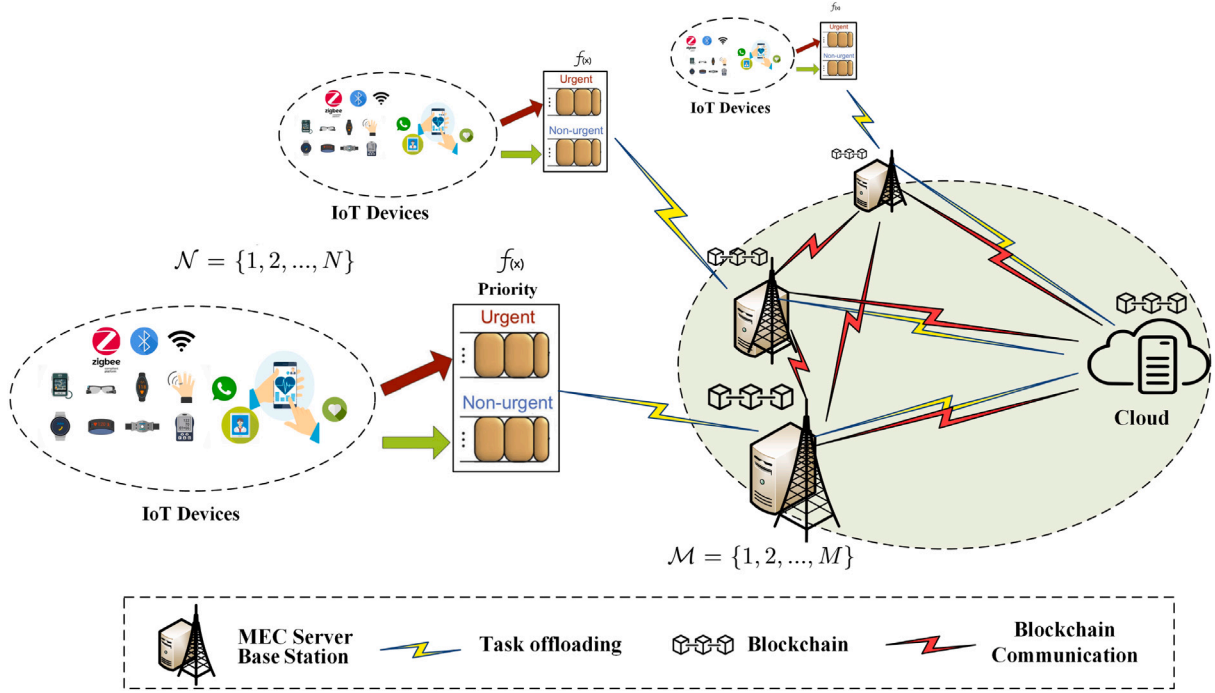


Fig. 1. System architecture for blockchain-based computational offloading in IoT network.

$\mathbf{X}$; i.e., each task is assigned to only one MEC server or the cloud. The block-manager row has a single “1” among MEC columns, and the verifier row may contain one or more “1”s. Resource feasibility requires $\sum_{n=1}^{N+2} f_{n,m} \leq F_m^{\max}$ for each server m (and similarly for the cloud), and consistency enforces $f_{n,m} = 0$ whenever $x_{n,m} = 0$. The key notations are summarized in Table 2.

$$\mathbf{X} = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,M} & x_{1,M+1} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,M} & x_{2,M+1} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{N,1} & x_{N,2} & \cdots & x_{N,M} & x_{N,M+1} \\ x_{N+1,1} & x_{N+1,2} & \cdots & x_{N+1,M} & x_{N+1,M+1} \\ x_{N+2,1} & x_{N+2,2} & \cdots & x_{N+2,M} & x_{N+2,M+1} \end{bmatrix} \begin{array}{l} \text{IoT Device 1} \\ \text{IoT Device 2} \\ \vdots \\ \text{IoT Device } N \\ \text{Block Manager} \\ \text{Verifier} \end{array} \quad (1)$$

$$\mathbf{F} = \begin{bmatrix} f_{1,1} & f_{1,2} & \cdots & f_{1,M} & f_{1,M+1} \\ f_{2,1} & f_{2,2} & \cdots & f_{2,M} & f_{2,M+1} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ f_{N,1} & f_{N,2} & \cdots & f_{N,M} & f_{N,M+1} \\ f_{N+1,1} & f_{N+1,2} & \cdots & f_{N+1,M} & f_{N+1,M+1} \\ f_{N+2,1} & f_{N+2,2} & \cdots & f_{N+2,M} & f_{N+2,M+1} \end{bmatrix} \begin{array}{l} \text{IoT Device 1} \\ \text{IoT Device 2} \\ \vdots \\ \text{IoT Device } N \\ \text{Block Manager} \\ \text{Verifier} \end{array} \quad (2)$$

3.1. Task offloading model

The system consists of N IoT devices, denoted by the set $\mathcal{N} = \{1, 2, \dots, N\}$, and M MEC servers, denoted by the set $\mathcal{M} = \{1, 2, \dots, M\}$. Each task $n \in \mathcal{N}$ generated by an IoT Device is characterized by a tuple $\langle D_n, C_n, l_n \rangle$, where D_n is the size of the input data (in bits), C_n is the computational workload (in CPU cycles required for completion), and l_n is the task's maximum tolerable delay, or deadline (in seconds). This deadline parameter inherently categorizes the incoming workload into varying levels of priority, effectively creating the urgent and non-urgent task flows depicted in the system architecture (Fig. 1). By incorporating l_n into the optimization constraints, the system ensures that delay-sensitive (urgent) tasks are prioritized appropriately.

A flexible offloading strategy is employed where a task can be offloaded to either one of the M MEC servers or to the central cloud server (denoted as server $M + 1$). The decision is based on optimizing system-wide objectives such as latency and energy consumption.

3.2. Communication model

The wireless communication links between the IoT Devices and MEC servers are modeled. The uplink data transmission rate from IoT Device n to MEC server m is determined by the Shannon-Hartley theorem:

$$r_{nm} = B_{nm} \log_2 \left(1 + \frac{P_n |g_{nm}|^2}{\omega} \right) \quad (3)$$

where B_{nm} is the allocated channel bandwidth, P_n is the transmission power of IoT Device n , g_{nm} is the channel gain between them, and ω is the background noise power.

Similarly, the backhaul communication rate from MEC server m to the cloud server is given by:

$$r_m^c = B_m^c \log_2 \left(1 + \frac{P_m |g_m^c|^2}{\omega} \right) \quad (4)$$

where P_m is the transmission power of MEC server m and B_m^c is its backhaul bandwidth.

3.3. Computational offloading latency and energy model

The total time to execute task n on MEC server m consists of the transmission time and the computation time:

$$T_{nm}^{\text{MEC}} = \frac{D_n}{r_{nm}} + \frac{C_n}{f_{nm}} \quad (5)$$

If the task is offloaded to the cloud via MEC server m , the total time includes the ID-to-MEC transmission, MEC-to-cloud transmission, and cloud computation time:

$$T_n^{\text{Cloud}} = \frac{D_n}{r_{nm}} + \frac{D_n}{r_m^c} + \frac{C_n}{f_{n,M+1}} \quad (6)$$

Table 2
Nomenclature.

Notation	Description
$\mathcal{N} = \{1, \dots, N\}$	IoT devices (size N).
$\mathcal{M} = \{1, \dots, M\}$	MEC servers (size M).
$\langle D_n, C_n, \ell_n \rangle$	Task n : input D_n (bits), workload C_n (cycles), deadline ℓ_n (s).
B_{nm}	Uplink bandwidth from device n to MEC server m .
P_n	Transmission power of device n (W).
g_{nm}	Channel gain between device n and MEC server m .
r_{nm}	Uplink rate from device n to MEC server m (bit/s).
B_m^c	Backhaul bandwidth from MEC server m to cloud.
P_m	Transmission power of MEC server m (W).
g_m^c	Backhaul channel gain between MEC server m and cloud.
r_m^c	Backhaul rate from MEC server m to cloud (bit/s).
$\mathbf{F}$	CPU allocation matrix; $f_{n,m}$ is the CPU allocated by server m to task n (cycles/s).
$k_m^{\text{comp}}, k_c^{\text{comp}}$	CPU capacitance coefficients of MEC server m and cloud.
$W(\mathbf{X})$	Total reputation of selected blockchain nodes.
n_{Tr}	Number of transactions per block.
D_{Tr}	Average transaction size (bits).
C_{Tr}	CPU cycles per bit for block creation.
C_{ver}	CPU cycles required for verification.
O	Feedback size (bits).
r_d, r_u	Average downlink and uplink rates (bit/s).
ψ	Broadcast overhead factor.
p_m, p_v	Transmission power of MEC server m and verifier v (W).
F_m, F_c	Maximum CPU capacity of MEC server m and cloud (cycles/s).
S_T	Slot length (s).
V	Number of verifiers.

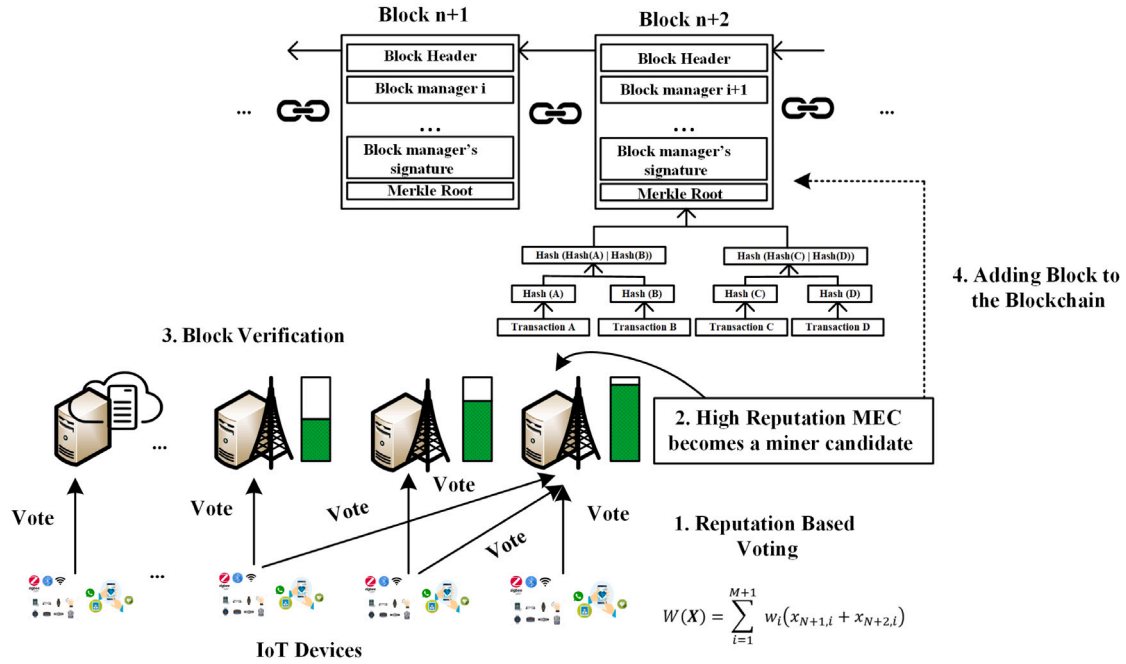


Fig. 2. Workflow of the Enhanced Delegated Proof of Stake (EDPoS) consensus mechanism used in the IoT network.

Thus, the total offloading latency for task n , T_n^{Off} , is:

$$T_n^{\text{Off}}(\mathbf{X}, \mathbf{F}) = \sum_{m=1}^M x_{n,m} T_{nm}^{\text{MEC}} + x_{n,M+1} T_n^{\text{Cloud}} \quad (7)$$

The energy consumed by the IoT Device for transmitting the task to an MEC server is $E_{n,m}^{\text{tx}} = P_n \frac{D_n}{r_{nm}}$. The energy consumed by the MEC server for computation is $E_{n,m}^{\text{comp}} = k_m^{\text{comp}} (f_{nm})^2 C_n$, where k_m^{comp} is the effective switched capacitance coefficient of the server's CPU architecture [6]. The total energy for MEC offloading is:

$$E_{nm}^{\text{MEC}} = P_n \frac{D_n}{r_{nm}} + k_m^{\text{comp}} (f_{nm})^2 C_n \quad (8)$$

For cloud offloading, the energy includes IoT Device transmission, MEC transmission, and cloud computation:

$$E_n^{\text{Cloud}} = P_n \frac{D_n}{r_{nm}} + P_m \frac{D_n}{r_m^c} + k_c^{\text{comp}} (f_{n,M+1})^2 C_n \quad (9)$$

The total offloading energy for task n , E_n^{Off} , is:

$$E_n^{\text{Off}}(\mathbf{X}, \mathbf{F}) = \sum_{m=1}^M x_{n,m} E_{nm}^{\text{MEC}} + x_{n,M+1} E_n^{\text{Cloud}} \quad (10)$$

3.4. Blockchain system model

An Enhanced Delegated Proof of Stake (EDPoS) consensus mechanism is adopted, inspired by [52], which is more energy-efficient and

scalable for IoT environments than Proof of Work. As illustrated in Fig. 2, the workflow of the consensus mechanism can be divided into four major steps. First, the network performs a reputation-based voting operation. Second, using the reputation information, MEC servers with high reputations are selected as candidates for the role of block manager (miner). Third, after the selected candidate creates the block, the verifier nodes verify it. Finally, the created block is added to the blockchain.

The considered blockchain layer will enhance the trustworthy collaboration between MEC/cloud nodes by offering a tamper-resistant way to record the task result, a decentralized verification system, and an auditable history of the services rendered. Some of the threats addressed in this work are unreliable or selfish nodes, inconsistent verifier verification behavior, verifier-node failures, and tampering or disputes of task-result records since execution. The blockchain ledger helps overcome these challenges by maintaining the metadata and verification data for the tasks in an append-only manner, while using the reputation-aware EDPoS mechanism to lower the possibility of selecting low-reputation nodes to perform the critical consensus tasks. Hence, the security analysis does not aim at giving a comprehensive cryptographic proof of security against all potential attacks by an adversary, but rather on the data integrity, the auditability, the reliability of the consensus, and the trusted selection of the roles.

In this model, MEC servers and the cloud server are candidates to become block managers or verifiers. The selection is based on a reputation score, which is modeled separately from the instantaneous computational capacity of each node. In this work, the reputation score reflects trust-related factors such as network reliability, uptime, successful verification history, failure behavior, and historical service consistency, whereas computational capability is captured independently through the CPU-frequency allocation variables and capacity constraints. The total reputation of the selected block manager and verifiers is a key objective to maximize:

$$W(\mathbf{X}) = \sum_{i=1}^{M+1} w_i(x_{N+1,i} + x_{N+2,i}) \quad (11)$$

where w_i is the reputation score of node i . Since $x_{N+1,i}$ and $x_{N+2,i}$ are binary decision variables, this objective function strictly aggregates the total reputation of the chosen nodes. The exact number of block managers and verifiers is explicitly controlled by constraints C_3 and C_4 in Section 4.

The total latency of the blockchain process, T^B , for creating and validating one block is the sum of latencies from several stages: block generation (T^{generate}), transmission to verifiers (T^{transmit}), parallel verification (T^{verify}), broadcasting of verification results ($T^{\text{broadcasting}}$), and feedback to the block manager (T^{feedback}).

$$T^B(\mathbf{X}, \mathbf{F}) = T^{\text{gen}} + T^{\text{tx}} + T^{\text{ver}} + T^{\text{bc}} + T^{\text{fb}}, \quad (12)$$

where

$$T^{\text{gen}} = \sum_{m=1}^{M+1} x_{N+1,m} \frac{n_{\text{Tr}} D_{\text{Tr}} C_{\text{Tr}}}{f_{N+1,m}}, \quad (13)$$

$$T^{\text{tx}} = \frac{n_{\text{Tr}} D_{\text{Tr}}}{r^d}, \quad (14)$$

$$T^{\text{ver}} = \max_{v: x_{N+2,v}=1} \left\{ \frac{C_{\text{ver}}}{f_{N+2,v}} \right\}, \quad (15)$$

$$T^{\text{bc}} = \frac{\psi n_{\text{Tr}} D_{\text{Tr}}}{r^d} \sum_{v=1}^{M+1} x_{N+2,v}, \quad (16)$$

and

$$T^{\text{fb}} = \frac{O}{r^u}. \quad (17)$$

This decomposition explicitly separates block generation, block transmission, parallel verification, broadcasting, and feedback propagation.

where n_{Tr} is the number of transactions per block, D_{Tr} is the average transaction size, C_{Tr} is the processing cycles per bit for block generation, C_{ver} is the total cycles for verification, O is the feedback message size, and r^d, r^u are average downlink/uplink rates within the edge network.

Similarly, the total energy consumed by the blockchain process, E^B , is the sum of energy consumed in each stage:

$$E^B(\mathbf{X}, \mathbf{F}) = E^{\text{gen}} + E^{\text{tx}} + E^{\text{ver}} + E^{\text{bc}} + E^{\text{fb}}, \quad (18)$$

where

$$E^{\text{gen}} = \sum_{m=1}^{M+1} x_{N+1,m} k_m^{\text{comp}} f_{N+1,m}^2 n_{\text{Tr}} D_{\text{Tr}} C_{\text{Tr}}, \quad (19)$$

$$E^{\text{tx}} = \frac{n_{\text{Tr}} D_{\text{Tr}}}{r^d} \sum_{m=1}^{M+1} p_m x_{N+1,m}, \quad (20)$$

$$E^{\text{ver}} = \sum_{v=1}^{M+1} x_{N+2,v} k_v^{\text{comp}} f_{N+2,v}^2 C_{\text{ver}}, \quad (21)$$

$$E^{\text{bc}} = \frac{\psi n_{\text{Tr}} D_{\text{Tr}}}{r^d} \sum_{v=1}^{M+1} p_v x_{N+2,v}, \quad (22)$$

and

$$E^{\text{fb}} = \frac{O}{r^u} \sum_{v=1}^{M+1} p_v x_{N+2,v}. \quad (23)$$

Where, E^{gen} represents the energy consumed for block generation, E^{tx} is the energy consumed for transmission, E^{ver} is the energy consumed for block verification, E^{bc} is the energy consumed for disseminating the verification results, and E^{fb} is the energy consumed for sending the verification feedback. The coefficients k_m^{comp} and k_v^{comp} denote the computational energy factors of the block producer and verifier nodes, respectively. The parameters p_m and p_v indicate the transmit power of MEC server m and of the v th verifier, respectively.

4. Problem formulation

To jointly analyze task offloading and blockchain processing under a synchronized decision horizon, the system is modeled in discrete time slots of duration S_T . This slot-based representation is appropriate because both the offloading of a task batch and the processing of a blockchain block consume shared communication and computational resources over a bounded operational interval. Under steady-state pipelined operation, the effective latency per slot is determined by the slower of these two parallel processes.

$$T^{\text{Tot}}(\mathbf{X}, \mathbf{F}) = \max_{n \in \mathcal{N}} \left(T_n^{\text{Off}}(\mathbf{X}, \mathbf{F}), T^B(\mathbf{X}, \mathbf{F}) \right) \quad (24)$$

This total latency, T^{Tot} , is used to determine the feasible time slot duration S_T for the current workload batch and is the main objective to be minimized in Problem P1. Note that decreasing the duration of the time slots S_T also reduces the overall worst-case system latency. The total system energy consumption per time slot is the sum of the offloading energy for all tasks and the blockchain processing energy.

$$E^{\text{Tot}}(\mathbf{X}, \mathbf{F}) = \sum_{n \in \mathcal{N}} E_n^{\text{Off}}(\mathbf{X}, \mathbf{F}) + E^B(\mathbf{X}, \mathbf{F}) \quad (25)$$

The primary goal is to find the optimal task allocation matrix $\mathbf{X}$ and resource allocation matrix $\mathbf{F}$ that solve the following multi-objective optimization problem (P1).

$$\text{P1: } \min_{\mathbf{X}, \mathbf{F}} S_T \quad (26a)$$

$$\min_{\mathbf{X}, \mathbf{F}} E^{\text{Tot}}(\mathbf{X}, \mathbf{F}) \quad (26b)$$

$$\max_{\mathbf{X}, \mathbf{F}} W(\mathbf{X}) \quad (26c)$$

subject to:

$$C_1 : x_{n,m} \in \{0, 1\}, \quad \forall n \in \{1, \dots, N+2\}, \forall m \quad (26d)$$

$$C_2 : \sum_{m=1}^{M+1} x_{n,m} = 1, \quad \forall n \in \mathcal{N} \quad (26e)$$

$$C_3 : \sum_{m=1}^{M+1} x_{N+1,m} = 1 \quad (26f)$$

$$C_4 : \sum_{m=1}^{M+1} x_{N+2,m} = V \quad (26g)$$

$$C_5 : \sum_{n=1}^N x_{n,m} f_{n,m} + x_{N+1,m} f_{N+1,m} + x_{N+2,m} f_{N+2,m} \leq F_m, \quad \forall m = 1, \dots, M \quad (26h)$$

$$C_6 : \sum_{n=1}^N x_{n,M+1} f_{n,M+1} + x_{N+1,M+1} f_{N+1,M+1} + x_{N+2,M+1} f_{N+2,M+1} \leq F_c \quad (26i)$$

$$C_7 : T_n^{\text{Off}}(\mathbf{X}, \mathbf{F}) \leq l_n - s_b, \quad \forall n \in \mathcal{N} \quad (26j)$$

$$C_8 : T^B(\mathbf{X}, \mathbf{F}) \leq S_T \quad (26k)$$

$$C_9 : T_n^{\text{Off}}(\mathbf{X}, \mathbf{F}) \leq S_T \quad (26l)$$

$$C_{10} : n_{T_r} = \min(n_{T_r}^{\max}, n_{T_r}^{\text{Total}}) \quad (26m)$$

Constraint (26d) defines the decision variables as binary. (26e) ensures each task is offloaded to exactly one server. (26f) ensures exactly one block manager is selected. (26g) sets the number of verifiers to a constant V . (26h) and (26i) represent the total computational capacity constraints for each MEC server and the cloud server, respectively, ensuring that the allocated CPU frequencies do not exceed the available capacity. (26j) is the task deadline constraint. (26k) and (26l) ensure that blockchain and offloading latencies are within the time slot duration S_T . (26m) specifies the permissible range for the number of transactions in each block.

For cases where a single solution is desired, a weighted-sum single-objective problem is formulated by normalizing and combining the objectives into a unified cost function:

$$\min_{\mathbf{X}, \mathbf{F}} \text{Cost} = \alpha \frac{E^{\text{Tot}}}{E_{\text{norm}}} + \beta \frac{S_T}{S_{T,\text{norm}}} - (1 - \alpha - \beta) \frac{W}{W_{\text{norm}}} \quad (27)$$

where $\alpha, \beta \in [0, 1]$, $\alpha + \beta \leq 1$ are weighting factors which illustrate the relative importance of each objective. The normalization constants E_{norm} , $S_{T,\text{norm}}$, and W_{norm} are static scaling factors for each simulation scenario, and are calculated based on maximum or typical values of the energy feasible range, the latency feasible range, and the reputation range, respectively. These constants are constants and do not change during optimization. They are only used to allow the objectives to be compared in comparable numbers, and not because they have relationships with each other in physical units.

5. Proposed solution

The formulated optimization problem P1 is an MINLP problem. The combination of binary decision variables (for allocation) and continuous variables (for resource distribution), along with non-linear objective functions and constraints, makes this problem NP-hard [53]. Exact solvers are computationally intractable for even moderately sized networks. Therefore, two solution methodologies based on powerful meta-heuristic evolutionary algorithms are proposed (Fig. 3).

5.1. NSGA-II for multi-objective optimization

The NSGA II is a well-established elitist multi-objective evolutionary algorithm used to find a set of non-dominated solutions, known as the Pareto front [54]. It is particularly suitable for this problem, as it can identify the optimal trade-offs among latency, energy consumption, and reputation. The implementation is detailed in Algorithm 1 and comprises the following core components.

1. **Chromosome Representation:** Each chromosome represents a potential solution, encoding the task allocation matrix $\mathbf{X}$ and the resource allocation matrix $\mathbf{F}$. These matrices define which server handles each task, the designation of the block manager and verifiers, and the allocated CPU frequencies.
2. **Initialization:** An initial population of size P is generated randomly, ensuring that each individual solution complies with all system constraints (e.g., C1–C5).
3. **Fitness Evaluation:** For each individual, the three objective functions latency (S_T), total energy (E^{Tot}), and reputation (W) are calculated.
4. **Non-Dominated Sorting:** The population is partitioned into non-domination levels (fronts). The first front (F_1) contains solutions that are not dominated by any other solution. The second front (F_2) contains solutions dominated only by those in F_1 , and so on.
5. **Crowding Distance:** To maintain diversity and ensure a well-distributed Pareto front, a crowding distance is calculated for each solution within a front. This metric estimates the density of neighboring solutions, and individuals in less crowded regions are prioritized.
6. **Selection:** A binary tournament selection operator is employed, which prioritizes solutions based on their non-domination rank. If two solutions have the same rank, the one with the larger crowding distance is selected.
7. **Genetic Operators:** Offspring are generated by applying crossover and mutation:
 - **Crossover:** Uniform crossover is applied to the binary allocation matrix $\mathbf{X}$, and a simulated binary crossover (SBX) is used for the continuous resource allocation values in $\mathbf{F}$.
 - **Mutation:** Random alterations are introduced into offspring. For $\mathbf{X}$, this involves flipping bits, while for $\mathbf{F}$, polynomial mutation adjusts the CPU frequency values.
8. **Elitism:** The parent and offspring populations are combined. The best P individuals are selected for the next generation based on their non-domination rank and crowding distance, which guarantees that the best-found solutions are preserved across generations.
9. **Termination:** The algorithm runs for a fixed number of generations (G). The final output is the first Pareto front (F_1), which contains the set of optimal trade-off solutions.

5.2. Genetic algorithm for single-objective optimization

For the single-objective formulation, which minimizes the weighted-sum cost function in (27), a standard GA is employed. This formulation allows the network operator to explicitly encode the relative priorities of latency, energy consumption, and reputation through the weighting coefficients α and β . The overall GA procedure is summarized in Algorithm 2. The proposed GA adopts the same chromosome representation, population initialization strategy, and genetic operators (i.e., crossover and mutation) as the NSGA-II framework.

In contrast to NSGA-II, however, the fitness evaluation and parent-selection mechanisms are simplified due to the scalar nature of the objective function. Specifically, the fitness of each individual is computed as the inverse of its cost value defined in (27). Parent selection is then carried out using standard selection schemes, such as tournament selection or roulette-wheel selection, in which candidate solutions are chosen directly according to this scalar fitness value. Elitism is additionally incorporated to retain the best solution identified in each generation. After G generations, the algorithm outputs the best feasible solution that minimizes the weighted-sum cost function for the specified parameter setting.

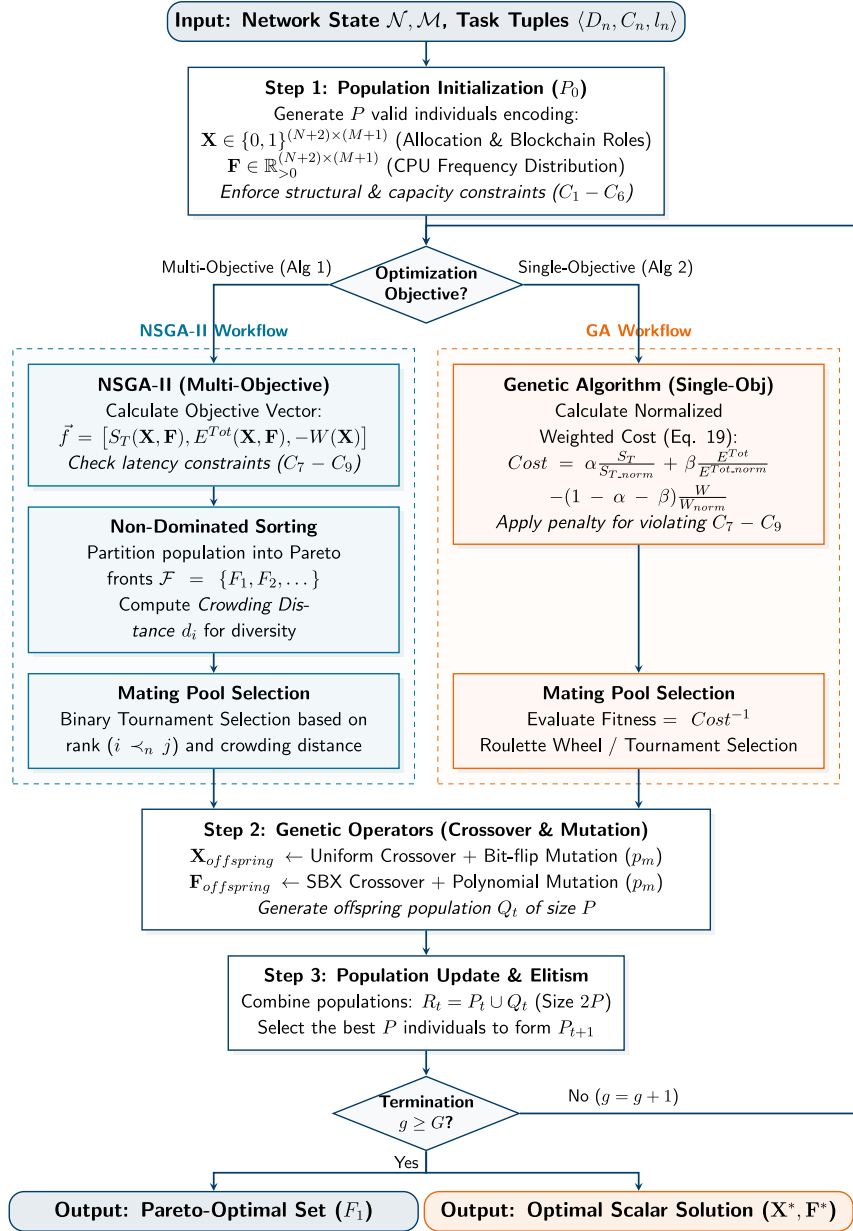


Fig. 3. NSGA-II and GA Flowchart for Task Offloading in BloT Network Optimization.

5.3. Computational complexity analysis

Let P denote the population size, G the number of generations, Q the number of objectives, and

$$D = (N + 2)(M + 1) \quad (28)$$

the main chromosome dimension associated with the assignment and resource-allocation matrices. For NSGA-II, the non-dominated sorting requires $O(QP^2)$ operations per generation and crowding-distance calculation requires $O(QP \log P)$. The crossover, mutation, repair, and objective-evaluation operations require $O(PD + PC_{eval})$ for a generation, where C_{eval} is the cost of computing latency, energy, reputation, and constraint violations for one individual. So the total complexity of the NSGA-II based method is:

$$O(G(QP^2 + QP \log P + PD + PC_{eval})). \quad (29)$$

For the GA-based scalarized formulation, selection and elitist replacement require at most $O(P \log P)$ per generation, while crossover,

mutation, repair, and cost evaluation require $O(PD + PC_{eval})$. Hence, the total complexity of the GA-based method is

$$O(G(P \log P + PD + PC_{eval})). \quad (30)$$

These expressions show that NSGA-II is more computationally expensive due to non-dominated sorting, whereas GA is more suitable when a single scalarized operating point is sufficient.

5.4. Implementation considerations

The proposed framework is intended for periodic network-level resource orchestration rather than per-packet scheduling. For practical applications, the NSGA-II component can be run in an offline manner or on longer intervals to derive Pareto-optimal operating points under typical network conditions. At the online stage, the network administrator can choose from among candidate solutions based on current needs regarding latency, energy consumption, and trust metrics. The formulation based on GA scalarization can also be considered when the user sets certain preference weights.

Algorithm 1: NSGA-II for Multi-Objective Task Offloading in BIoT Network

Input : Network parameters; Algorithm parameters (Population size P , Generations G , Crossover probability p_c , Mutation probability p_m).

Output: Pareto-optimal front F_1 minimizing S_T , E^{Tot} and maximizing W .

- 1 **Step 1: Initialization;**
- 2 Generate an initial population of size P with random, valid task and resource allocations;
- 3 Evaluate objectives (S_T , E^{Tot} , W) for each individual;
- 4 **for** $g \leftarrow 1$ **to** G **do**
- 5 **Step 2: Non-Dominated Sorting;**
- 6 Classify the population into Pareto fronts $F_1, F_2, \dots$;
- 7 **Step 3: Crowding Distance Calculation;**
- 8 Compute the crowding distance for every individual in each front;
- 9 **Step 4: Selection;**
- 10 Create a mating pool using crowded tournament selection based on rank and crowding distance;
- 11 **Step 5: Genetic Operators;**
- 12 Apply crossover and mutation operators to the mating pool to generate an offspring population;
- 13 **Step 6: Population Update;**
- 14 Combine parent and offspring populations;
- 15 Select the best P individuals for the next generation using non-dominated sorting and crowding distance (elitism);
- 16 **Step 7: Output;**
- 17 Return the final non-dominated front F_1 ;

Algorithm 2: Genetic Algorithm for Single-Objective Task Offloading

Input : Network parameters; Algorithm parameters (Population size P , Generations G , Crossover probability p_c , Mutation probability p_m); Weights α, β .

Output: The solution (X, F) that minimizes the **Cost** function.

- 1 **Step 1: Initialization;**
- 2 Generate an initial population of size P with random, valid allocations;
- 3 Evaluate the **Cost** for each individual;
- 4 **for** $g \leftarrow 1$ **to** G **do**
- 5 **Step 2: Selection;**
- 6 Select parents from the population based on their fitness (inverse of **Cost**);
- 7 **Step 3: Genetic Operators;**
- 8 Apply crossover and mutation to the selected parents to produce an offspring population;
- 9 **Step 4: Population Update;**
- 10 Combine parent and offspring populations;
- 11 Retain the top P individuals with the best fitness for the next generation (elitism);
- 12 **Step 5: Output;**
- 13 Return the best solution found that minimizes **Cost**;

The main implementation overheads include collecting task meta-data, estimating channel states, updating node reputation scores, and exchanging blockchain coordination messages. Overheads associated with the optimization are practical in terms of MEC-enabled IoT networks, as the optimization takes place at MEC or cloud orchestration

Table 3
Simulation Parameters.

Parameter	Value
Channel bandwidth (B_{nm}, B_m^e)	20 MHz
IoT Device transmit power (P_r)	0.5 W
MEC server transmit power (P_m)	2 W
Noise power density (ω_0)	10^{-17} W/Hz
Energy coefficient ($k_m^{\text{comp}}, k_c^{\text{comp}}$)	10^{-26}
Task data size (D_r)	Uniform in [1, 5] Mb
Task workload (C_r)	Uniform in [0.5, 2] Gcycles
Transaction size (D_{tr})	2 Kb
Block size	1 Mb
Verification cycles (C_{ver})	10^7 cycles
Number of verifiers (V)	6
Population size (P)	100
Number of generations (G)	500
Crossover/mutation probability	0.9/0.1

nodes, but not on IoT devices themselves. However, in highly dynamic conditions, care must be taken that the execution time is lower than the length of a control slot.

6. Simulation results and analysis

Extensive simulations were conducted using Python to evaluate the performance of the proposed framework. The simulation parameters, chosen based on typical values from related literature [6,27,47], are summarized in Table 3. These values are also suitable for the considered blockchain-enabled IoT-MEC setting because they reflect realistic communication, computation, and verification scales. The simulation parameters are selected based on existing standard values for MEC and IoT offloading literature. Particularly, the effective switched capacitance is assumed to be 10^{-26} . This is one of the values used in the modeling of the CPU frequency-based energy consumption of the MEC system. The values of task data size, workload, wireless bandwidth, and power consumed in transmitting these tasks have been determined based on the typical scenario of IoT computation offloading systems. The values of the blockchain system are chosen accordingly.

A network with $M = 10$ MEC servers and one cloud server is considered. The number of IoT tasks N is varied from 10 to 40 to simulate different network loads. For heterogeneous resource scenarios, a reputation vector of $W = [2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 1]$ and a corresponding processing capacity vector of $F_m = [0.4, 0.5, 0.7, 0.9, 1, 2, 3, 4, 5, 6, 20]$ GHz are assumed.

The experimental evaluation is organized to progressively examine different aspects of the proposed framework. First, the Pareto-front results show the intrinsic trade-off between latency and energy across various network loads and processing capacities. Second, the single objective analysis examines the impact of the preference weights on the selected operating point. Third, the reputation and computational-capacity analyses make the effects of the blockchain-role selection and available processing resources clear. Fourth, the comparative baseline evaluation shows the overall performance gain of the proposed joint optimization framework. Finally, the contribution of each component, their robustness under different conditions, their reliability of consensus, and the practical implementation overhead are analyzed by the ablation, sensitivity, dynamic-channel, verifier-failure, and runtime analysis.

Fig. 4 presents the Pareto optimal fronts obtained by the multi-objective optimization algorithm, which demonstrates the tradeoff between total system latency and energy consumption. The analysis assumes three network load cases, as represented by the number of user devices ($N \in \{10, 20, 40\}$), and four cases of fixed processing capacities for the MEC servers ($F_m \in \{1, 5\}$ GHz). Each point on the curve represents the set of non-dominated solutions, meaning that improving one objective results in the degradation of the other objective. The

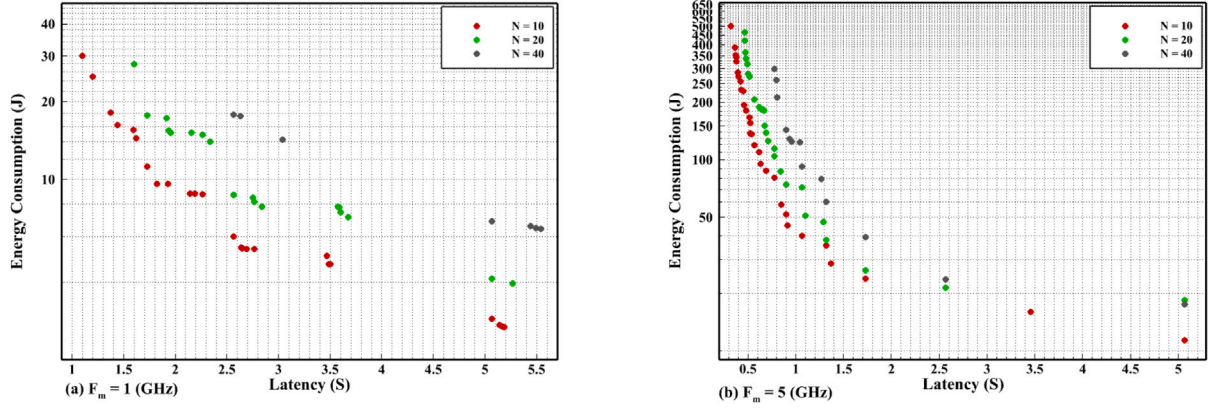


Fig. 4. Pareto fronts for latency vs. energy with varying task numbers (N) and maximum MEC processing frequencies ($F_{\max}$). (a) $F_{\max} = 1$ GHz, (b) $F_{\max} = 5$ GHz.

analysis of the obtained fronts provides the following key insights on the system's performance:

1. Impact of Network Load (N): For any fixed processing frequency, increasing the number of tasks from $N = 10$ to $N = 40$ causes the Pareto front to shift upwards and to the right. This indicates that a higher network load results in a degradation of achievable performance. At a constant latency, the total energy consumption increases with N , as more tasks require offloading and processing. Concurrently, for a fixed energy budget, the minimum achievable latency increases with N due to greater resource contention.

2. Impact of Processing Frequency (F_m): A significant change in the performance envelope is observed as the server frequency increases from 1 GHz (Fig. 4a) to 5 GHz (Fig. 4b).

- **Latency Reduction:** The minimum latency that is attainable reduces considerably. At $F_m = 1$ GHz, the minimum latency that is attainable is about 1 s. On the other hand, for an F_m of 5 GHz, it is clear that latencies that are less than 0.5 s are attainable. This demonstrates the direct performance benefit that arises from more powerful computing resources.
- **Energy Cost:** This increase in speed comes at the cost of energy consumption. At $F_m = 1$ GHz, the energy consumption that is attainable is about 40 Joules. On the other hand, at $F_m = 5$ GHz, the energy consumption that is attainable is between 500 and 550 Joules. This highlights a critical trade-off. High-frequency servers are faster, but they are significantly less energy-efficient when performing the same tasks.

From the results presented, it is evident that an increase in the available processing frequency $F_{\max}$ from 1 GHz to 5 GHz causes a significant shift downward and to the left of the Pareto front, indicating that lower latency and energy values are simultaneously achievable. For instance, with $N = 40$ tasks, the minimum achievable latency drops from over 2 s at 1 GHz to under 0.5 s at 5 GHz. However, this improvement is associated with a significant increase in possible energy consumption. For any given frequency, an increase in the number of tasks N shifts the Pareto front to the right and upward, as a higher network load demands more resources and energy.

6.1. Single-objective performance analysis

Fig. 5 demonstrates the effect of the weighting factor α on the optimal latency and energy values achieved by the single-objective genetic algorithm. In this context, $\beta = 1 - \alpha$, and exclude the reputation component to focus on the traditional energy-latency trade-off.

As α increases, greater emphasis is placed on energy minimization. Consequently, the GA chooses solutions with reduced energy

consumption, which typically entails assigning lower CPU frequencies and sometimes offloading to more remote yet energy-efficient servers, resulting in increased delay (Fig. 5(a)). Conversely, when α approaches 0, the emphasis transitions to minimizing delay, leading to assertive resource allocation and consequently increased energy consumption (Fig. 5(b)).

This trade-off is increasingly evident in systems with increased resource availability (higher $F_{\max}$). At $F_{\max} = 15$ GHz, changing α from 0.1 to 0.9 results in a reduction of energy consumption exceeding 60%, illustrating that the capacity for energy savings has been substantially improved in high-performance systems. These results confirm that the weighting factor α provides an effective and flexible mechanism for tuning the GA behavior according to application-specific QoS priorities, energy-latency requirements, and available system resource conditions.

6.2. Impact of blockchain node reputation (W)

The selection of reputable nodes to perform roles such as block manager and verifier is critical for security and performance. As defined in Section 3.4, the total reputation W is the aggregate of the individual reputation scores (w_i) of the selected nodes. In the heterogeneous simulation setup, some high-reputation nodes also have higher processing capacities; therefore, the discussion distinguishes the effect of computational capacity on latency from the effect of reputation on trustworthy blockchain-role selection. Fig. 6 illustrates the Pareto fronts for two scenarios: one in which low-reputation nodes are selected ($W = 22$, total 5.1 GHz processing power for blockchain roles), and another in which high-reputation nodes are selected ($W = 62$, allocating 27 GHz). When high-reputation nodes are selected for blockchain tasks, the system achieves significantly lower latencies. This is because these powerful nodes can generate and verify blocks much faster, reducing the T^B component of the total latency. This suggests that a well-designed reputation system can directly improve system performance by intelligently allocating critical roles in the blockchain to the most capable nodes. It should be noted that reputation and computational capacity are conceptually different in the proposed formulation. Computational capacity directly affects block-generation and verification latency through the allocated CPU-frequency variables, while reputation affects the trustworthiness and reliability of blockchain role selection. Therefore, the reputation objective should not be interpreted as merely selecting high-computing-capacity nodes; rather, it guides the selection of more trustworthy consensus participants under the resource constraints of the system.

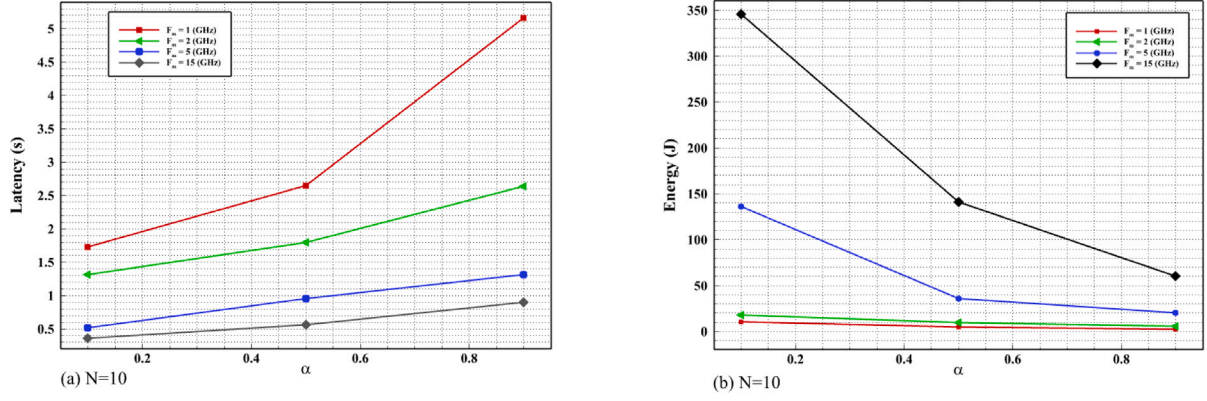


Fig. 5. Impact of the energy weight parameter α on (a) total system latency and (b) total system energy for different maximum processing frequencies.

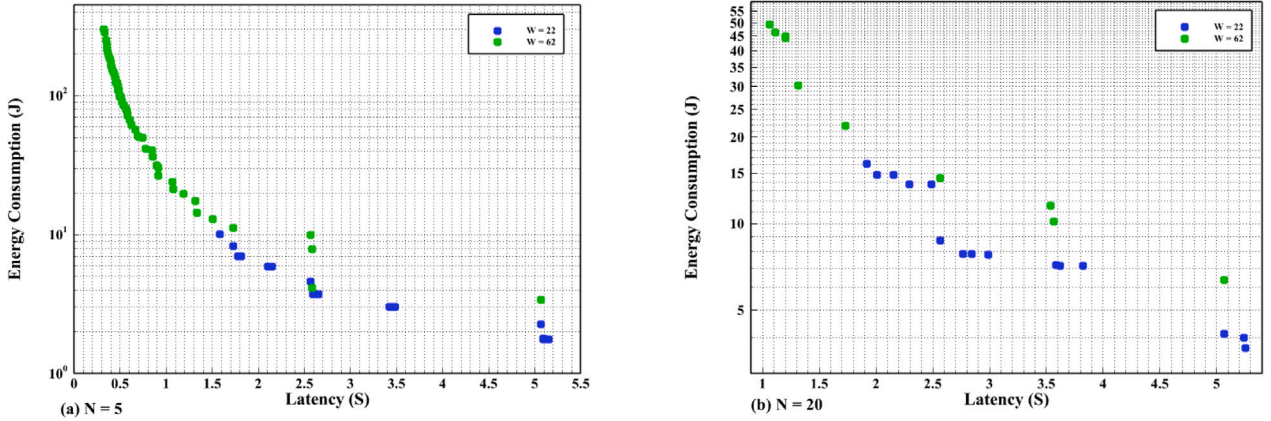


Fig. 6. Impact of total reputation (W) of blockchain nodes on the latency-energy Pareto front for $N = 5, 20$.

6.3. Impact of computational capacity and network load

To further analyze the system's behavior, Fig. 7 illustrates the minimum achievable latency as a function of the total computational capabilities (F_m) for varying network loads (N). From the results, it is evident that as computational capabilities increase, latency decreases. However, the rate of reduction is greater at lower frequency levels, indicating a diminishing rate of return as computational capabilities increase. At very high frequency levels, communication delay is a significant bottleneck. For a given level of computational capabilities, higher values of N result in higher latency, requiring adequate computational resources to handle large numbers of tasks.

6.4. Comparative analysis and convergence

The convergence behavior of the genetic algorithm is illustrated in Fig. 8 for a given scenario, where $N = 10$, $F_m = 2$ GHz, $\alpha = 0.1$. The cost function value is shown as a function of the number of iterations. Initially, the cost function value is high. However, as the algorithm iterates, it is observed that better configurations are found. Consequently, the cost function decreases significantly, and the cost function value is reduced before converging towards an optimal value. This trajectory verifies that the algorithm effectively navigates the solution space and converges on an improved solution.

6.5. Comparative baseline evaluation

The proposed framework is compared with five representative benchmark methods: Random Allocation, Greedy-Latency, Greedy-

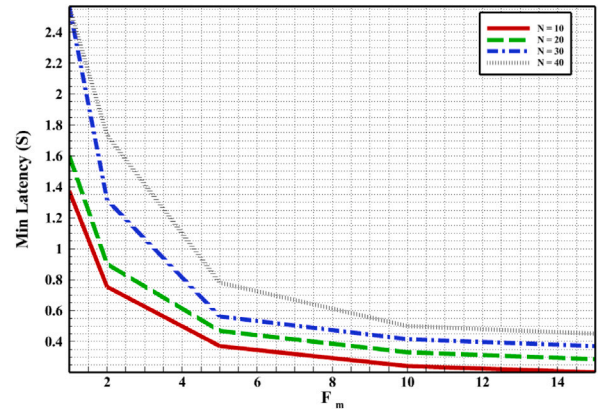


Fig. 7. Impact of maximum MEC computational capacity ($F_{\max}$) on minimum achievable latency for different network loads (N).

Energy, Reputation-Only Selection, and PSO-based optimization. Random Allocation assigns tasks and blockchain roles randomly among feasible nodes and is used as a lower-bound benchmark. Greedy-Latency assigns each task to the node that provides the lowest immediate execution latency, whereas Greedy-Energy assigns each task to the node that provides the lowest immediate energy consumption. Reputation-Only Selection assigns blockchain management roles mainly according to node reputation, while task offloading and CPU-frequency allocation are optimized separately. PSO-based optimization is included as a representative meta-heuristic benchmark because PSO has been widely

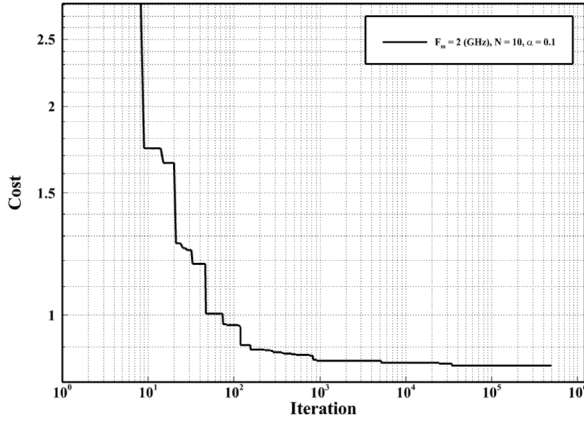


Fig. 8. Convergence of the GA cost function ($N = 10$, $F_m = 2$ GHz, $\alpha = 0.1$).

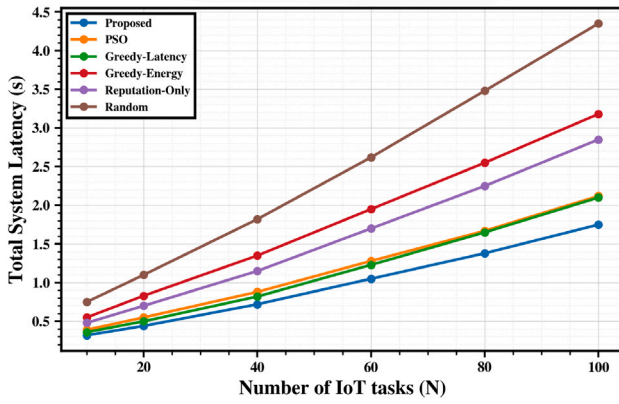


Fig. 9. Baseline comparison in terms of total system latency under different numbers of IoT tasks.

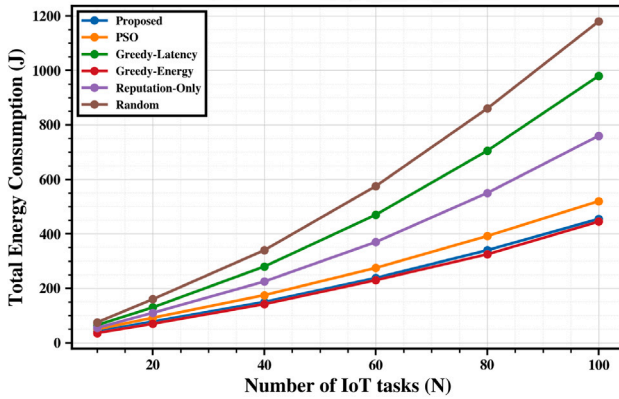


Fig. 10. Baseline comparison in terms of total energy consumption under different numbers of IoT tasks.

used for computation offloading and resource-allocation problems in MEC systems [32]. All benchmark methods are evaluated under the same network, workload, and blockchain settings.

Fig. 9 presents the comparison between the latency achieved by the proposed framework and Random Allocation, Greedy-Latency, Greedy-Energy, Reputation-Only Selection, and PSO-based optimization when the number of IoT tasks is increased. The latency experienced by all approaches grows as N increases since a greater number of tasks

implies higher levels of competition over available communication and computing resources. The highest value of latency is obtained by Random Allocation due to the lack of information about channel conditions, the computing capability of the server nodes, and the compatibility of selected blockchain roles. Greedy-Latency offers good latency values for light workloads, but as the value of N increases, its performance deteriorates due to the local nature of the decision making process without joint consideration of overhead and resource availability. On the other hand, Reputation-Only Selection enhances blockchain role selection without optimizing task offloading and CPU frequencies. PSO-based optimization offers better results compared to random and greedy techniques but does not attain the balanced latency-energy-reputation ratio of the proposed approach.

Fig. 10 presents the total energy consumed by different schemes according to the increase of task workload. As N increases, energy consumption is increasing due to more tasks needing to transmit and compute their results. Latency-oriented greedy scheme has higher energy consumption than the other schemes because the algorithm prefers to have tasks run at high frequencies to decrease latency. On the other hand, the energy-oriented greedy scheme results in reduced energy consumption; however, it results in increased latency, as depicted in Fig. 9. This confirms that the proposed joint optimization model can achieve a better latency-energy trade-off than single-objective or locally greedy strategies.

6.6. Coordination overhead metrics

To quantify the additional coordination cost introduced by the proposed framework, we define the control-message overhead per decision slot as

$$O_{\text{ctrl}} = N O_{\text{task}} + (M + 1) O_{\text{rep}} + V O_{\text{vote}} + O_{\text{fb}}, \quad (31)$$

where O_{task} denotes the metadata size required for each task request, O_{rep} is the reputation-update message size per blockchain node, O_{vote} denotes the voting/verification coordination message size per verifier, and O_{fb} is the feedback message size. We also report the algorithm execution time as a computational overhead metric:

$$T_{\text{alg}} = t_{\text{end}} - t_{\text{start}}. \quad (32)$$

A solution is considered suitable for online orchestration if $T_{\text{alg}} \leq T_{\text{ctrl}}$, where T_{ctrl} is the control interval.

Fig. 11 presents the ablation study of the proposed framework. The full model is compared with several reduced variants, including the model without the reputation objective, without explicit blockchain-overhead modeling, with separate optimization of offloading and blockchain-role assignment, with random verifier selection, and without the repair mechanism. Removing the reputation objective decreases the trustworthiness of selected blockchain management nodes. Ignoring blockchain overhead produces optimistic latency and energy values but does not reflect the actual cost of block generation, verification, broadcasting, and feedback. Separate optimization also degrades performance because task offloading and blockchain-role assignment compete for the same computational and communication resources. The random-verifier variant reduces the quality of consensus-role selection, while removing the repair mechanism increases infeasible or inefficient allocations. These results confirm that the performance improvement of the proposed framework comes from the joint integration of offloading, resource allocation, blockchain-overhead modeling, and reputation-aware role selection.

6.7. Sensitivity analysis

Fig. 12 illustrates the sensitivity of the scalarized objective function with respect to different combinations of energy weight α and latency weight β . The rest of the weight, $1 - \alpha - \beta$, is assigned to the reputation objective. We exclude infeasible combinations with $\alpha + \beta > 1$. The

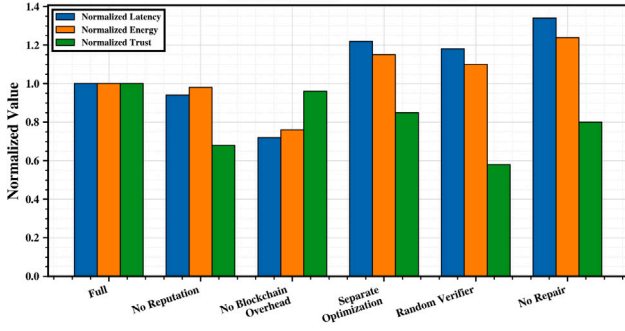


Fig. 11. Ablation study showing the impact of key components on normalized latency, energy consumption, and trustworthiness.

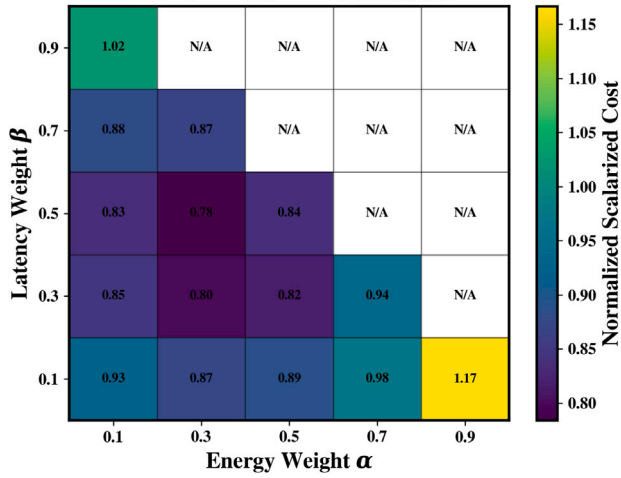


Fig. 12. Sensitivity of the normalized scalarized cost to different combinations of the energy weight α and latency weight β . Invalid combinations with $\alpha + \beta > 1$ are excluded.

results show that the operating point is sensitive to the choice of preference weights. As α increases, the energy saving becomes more important to the optimizer, which usually leads to the reduction of energy consumption but can increase the latency. When β increases, the optimizer cares more about reducing latency and tends to allocate more computational resources, which may increase energy consumption. The analysis shows that the proposed weighted-sum formulation provides a flexible mechanism to select application-specific operating points.

6.8. Dynamic channel and node-failure evaluation

Fig. 13 evaluates the robustness of the proposed framework under different channel conditions. The average SNR is varied to emulate poor, moderate, and favorable wireless environments. For all methods, latency decreases as SNR increases because a higher SNR improves the uplink transmission rate and reduces task-uploading delay. Under poor channel conditions, the random and greedy strategies experience larger latency degradation because they do not jointly adapt task assignment, computation allocation, and blockchain-role selection. The proposed framework maintains lower latency across different SNR levels by selecting more suitable computing nodes and allocating resources while accounting for both communication and blockchain-processing costs.

As illustrated in Fig. 14, it is shown that the consensus-failure probability increases with the increase in the verifier-node failure probability (p_f). With the increase in the probability of failure of

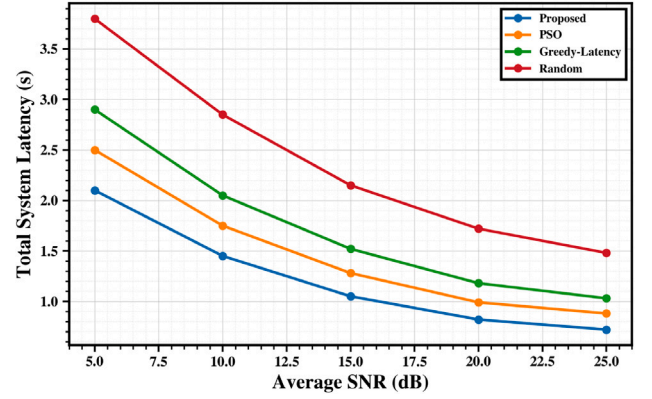


Fig. 13. Robustness evaluation under dynamic channel conditions: total system latency versus average SNR.

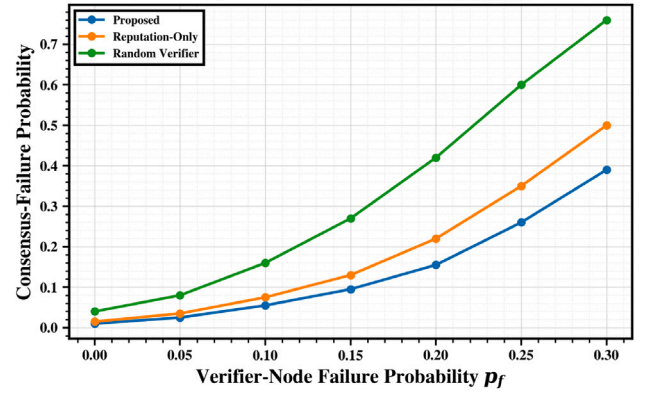


Fig. 14. Consensus-failure probability under different verifier-node failure probabilities.

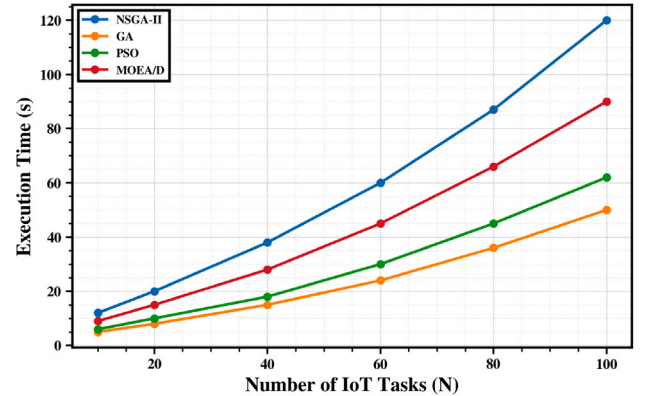


Fig. 15. Algorithm execution time under different numbers of IoT tasks.

the nodes, the likelihood of having insufficient numbers of verifier nodes increases, thereby increasing the probability of consensus failure. The random verification approach demonstrates poor resilience due to the potential selection of nodes with low reliability and capacity. Reputation-based alone performs better due to the preference of selecting high-reputation nodes without taking into account task offloading and resource allocation.

6.9. Computational and communication overhead

Fig. 15 reports the algorithm execution time as the number of IoT tasks increases. The runtime of all evolutionary methods increases with N because the chromosome dimension and objective-evaluation cost grow with the number of task-assignment and resource-allocation variables. NSGA-II requires higher execution time than the scalar GA because non-dominated sorting and crowding-distance calculation introduce additional computational cost. However, NSGA-II provides a Pareto set of trade-off solutions, which is useful for offline or periodic network-level orchestration. The GA-based scalar formulation is computationally lighter and is more suitable when the operator requires a single operating point. These results clarify the practical trade-off between solution diversity and computational overhead.

7. Summary and conclusions

This study examines the main problem associated with resource allocation in blockchain-based IoT networks. It suggests a system model with a comprehensive optimization framework achieves minimal latency and energy consumption while maximizing network reputation. The model holistically captures the interplay between computational offloading via MEC and the operational overheads of blockchain, including a detailed analysis of the consensus process costs.

From a holistic system optimization perspective, this work presents both an NSGA-II-based multi-objective and a GA-based single-objective approach. The NSGA-II algorithm is capable of identifying the Pareto-optimal front, which can give network operators clear ideas about fundamental trade-offs related to latency and energy. It can further facilitate well-informed, flexible decisions by system operators with respect to application-specific requirements. The simulation results verify the effectiveness of the proposed framework. For performance and energy cost, a quantifiable trade-off was realized. More importantly, it can be observed that by utilizing a reputation system in assigning high-capacity nodes to critical blockchain roles, system performance can be significantly enhanced. From these findings, it became evident that, for complex IoT systems, resource allocation policies should be balanced with an integrated perspective.

In the future, this framework can be extended by incorporating machine learning techniques to achieve real-time dynamic resource allocation. Another promising research direction is the integration of smart contracts that will automate the enforcement of allocation policies on the blockchain. Finally, specific large-scale IoT case studies, such as autonomous vehicle networks or smart city infrastructures, should be addressed through this framework in order to validate its performance and scalability in real-world scenarios.

CRediT authorship contribution statement

Nasim Nezhadsistani: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Naghme S. Moayedian:** Writing – review & editing, Supervision, Project administration, Methodology, Conceptualization. **Arash Bozorgchenani:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Conceptualization. **Burkhard Stiller:** Writing – review & editing, Supervision, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

- [1] J. Gubbi, R. Buyya, S. Marusic, M. Palaniswami, Internet of things (IoT): A vision, architectural elements, and future directions, *Future Gener. Comput. Syst.* (ISSN: 0167-739X) 29 (7) (2013) 1645–1660.
- [2] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, M. Ayyash, Internet of things: A survey on enabling technologies, protocols, and applications, *IEEE Commun. Surv. Tutor.* 17 (4) (2015) 2347–2376.
- [3] M. Chen, S. Mao, Y. Liu, Big data: A survey, *Mob. Netw. Appl.* 19 (2) (2014) 171–209.
- [4] Y. Qiao, Y. Xue, Y. Zhai, D. Zhang, B. Chen, A.V. Vasilakos, M.S. Hossain, S. Mumtaz, A secure and efficient sharing scheme for medical IoT data based on consortium blockchain, *IEEE Internet Things J.* 12 (23) (2025) 51422–51437.
- [5] Y. Mao, C. You, J. Zhang, K. Huang, K.B. Letaief, A survey on mobile edge computing: The communication perspective, *IEEE Commun. Surv. Tutor.* 19 (4) (2017) 2322–2358.
- [6] S. Tian, K. Xu, S. Xiang, X. Dai, Z. Xiao, L. Zeng, Task offloading and resource scheduling in mobile edge-cloud computing based on edge competition and task prediction, *IEEE Trans. Serv. Comput.* 18 (5) (2025) 3262–3275.
- [7] P. Ranaweera, A.D. Jurcut, M. Liyanage, Survey on multi-access edge computing security and privacy, *IEEE Commun. Surv. Tutor.* 23 (2) (2021) 1078–1124.
- [8] A. Kumar, A. Pal, A survey on computation offloading and current trends, *IEEE Access* 13 (2025) 185318–185356.
- [9] Q. He, Z. Xi, Z. Feng, Y. Teng, L. Ma, Y. Cai, K. Yu, Telemedicine monitoring system based on fog/edge computing: A survey, *IEEE Trans. Serv. Comput.* 18 (1) (2025) 479–498.
- [10] M. Mukherjee, L. Shu, D. Wang, Survey of fog computing: Fundamental, network applications, and research challenges, *IEEE Commun. Surv. Tutor.* 20 (3) (2018) 1826–1857.
- [11] D. Sabella, K. Maloor, N. Smith, M. Vanderveen, A. Kourtis, Edge computing cybersecurity standards: Protecting infrastructure and applications, *IEEE Access* 12 (2024) 185328–185335.
- [12] M. Belotti, N. Božić, G. Pujolle, S. Secci, A vademecum on blockchain technologies: When, which, and how, *IEEE Commun. Surv. Tutor.* 21 (4) (2019) 3796–3838.
- [13] Z. Zheng, S. Xie, H. Dai, X. Chen, H. Wang, An overview of blockchain technology: Architecture, consensus, and future trends, in: 2017 IEEE International Congress on Big Data (BigData Congress), 2017, pp. 557–564.
- [14] H.N. Dai, Z. Zheng, Y. Zhang, Blockchain for internet of things: A survey, *IEEE Internet Things J.* 6 (5) (2019) 8076–8094.
- [15] J. Singh, S. Rani, P. Kumar, Blockchain and smart contracts: Evolution, challenges, and future directions, *ICKECS*, in: 2024 International Conference on Knowledge Engineering and Communication Systems, vol. 1, 2024, pp. 1–5.
- [16] A. Dorri, S.S. Kanhere, R. Jurdak, P. Gauravaram, Blockchain for IoT security and privacy: The case study of a smart home, in: 2017 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops), 2017, pp. 618–623.
- [17] A. Reyna, C. Martín, J. Chen, E. Soler, M. Díaz, On blockchain and its integration with IoT. Challenges and opportunities, *Future Gener. Comput. Syst.* 88 (2018) 173–190.
- [18] D.K.J.B. Saini, S. Kumar, A. Bhatt, R. Gupta, K. Joshi, D. Siddharth, Blockchain-based IoT applications, platforms, systems and framework, in: 2023 14th International Conference on Computing Communication and Networking Technologies, ICCCNT, 2023, pp. 1–6.
- [19] O. Novo, Blockchain meets IoT: An architecture for scalable access management in IoT, *IEEE Internet Things J.* 5 (2) (2018) 1184–1195.
- [20] Y. Zuo, Exploring the synergy: AI enhancing blockchain, blockchain empowering AI, and their convergence across IoT applications and beyond, *IEEE Internet Things J.* 12 (6) (2025) 6171–6195.
- [21] J. Zhang, X. Hu, Z. Ning, E.C.H. Ngai, L. Zhou, J. Wei, J. Cheng, B. Hu, Energy-latency tradeoff for energy-aware offloading in mobile edge computing networks, *IEEE Internet Things J.* 5 (4) (2018) 2633–2645, <http://dx.doi.org/10.1109/JIOT.2017.2786343>.
- [22] L. Cui, C. Xu, S. Yang, J.Z. Huang, J. Li, X. Wang, Z. Ming, N. Lu, Joint optimization of energy consumption and latency in mobile edge computing for internet of things, *IEEE Internet Things J.* 6 (3) (2019) 4791–4803, <http://dx.doi.org/10.1109/JIOT.2018.2869226>.
- [23] X. Xu, X. Zhang, H. Gao, Y. Xue, L.T. Qi, W. Dou, Become: Blockchain-enabled computation offloading for IoT in mobile edge computing, *IEEE Trans. Ind. Inform.* 16 (6) (2020) 4187–4195, <http://dx.doi.org/10.1109/TII.2019.2936869>.
- [24] J. Feng, F.R. Yu, Q. Pei, X. Chu, J. Du, L. Zhu, Cooperative computation offloading and resource allocation for blockchain-enabled mobile-edge computing: A deep reinforcement learning approach, *IEEE Internet Things J.* 7 (7) (2020) 6214–6228, <http://dx.doi.org/10.1109/JIOT.2019.2961707>.

- [25] A. Bozorgchenani, F. Mashhadi, D. Tarchi, S. Salinas Monroy, Multi-objective computation sharing in energy and delay constrained mobile edge computing environments, *IEEE Trans. Mob. Comput.* 20 (10) (2021) 2992–3005, <http://dx.doi.org/10.1109/TMC.2020.2994232>.
- [26] P. Wang, K. Li, B. Xiao, K. Li, Multi-objective optimization for joint task offloading, power assignment, and resource allocation in mobile edge computing, *IEEE Internet Things J.* 9 (14) (2022) 11737–11748, <http://dx.doi.org/10.1109/JIOT.2021.3132080>.
- [27] D. C. Nguyen, P. N. Pathirana, M. Ding, A. Seneviratne, Secure computation offloading in blockchain based IoT networks with deep reinforcement learning, *IEEE Trans. Netw. Sci. Eng.* 8 (4) (2021) 3192–3208.
- [28] Z.-Y. Chai, X. Liu, Y.-L. Li, A computation offloading algorithm based on multi-objective evolutionary optimization in mobile edge computing, *Eng. Appl. Artif. Intell.* 121 (2023) 105966, <http://dx.doi.org/10.1016/j.engappai.2023.105966>.
- [29] A. Bozorgchenani, S. Maghsudi, D. Tarchi, E. Hossain, Computation offloading in heterogeneous vehicular edge networks: On-line and off-policy bandit solutions, *IEEE Trans. Mob. Comput.* 21 (12) (2022) 4233–4248.
- [30] T.X. Tran, D. Pompili, Joint task offloading and resource allocation for multi-server mobile-edge computing networks, *IEEE Trans. Veh. Technol.* 68 (1) (2019) 856–868.
- [31] H.T. Ciptaningtyas, A.M. Shiddiqi, D. Purwitasari, A systematic literature review of genetic algorithm-based approaches for cloud task scheduling, in: 2023 14th International Conference on Information & Communication Technology and System, ICTS, 2023, pp. 319–324.
- [32] D. Basu, V. Krishnakumar, U. Ghosh, R. Datta, DeepCare: Deep learning-based smart healthcare framework using 5g assisted network slicing, in: 2022 IEEE International Conference on Advanced Networks and Telecommunications Systems, ANTS, 2022, pp. 201–206.
- [33] Z. Shi, Z. Zhang, M. Dai, Z. Xia, H. Wen, F. Huang, Deep reinforcement learning-based task offloading for multi-user distributed edge computing, in: 2024 30th International Conference on Mechatronics and Machine Vision in Practice (M2VIP), 2024, pp. 1–6.
- [34] H. Hu, D. Wu, F. Zhou, S. Jin, R.Q. Hu, Dynamic task offloading in MEC-enabled IoT networks: A hybrid DDPG-d3qn approach, in: 2021 IEEE Global Communications Conference, GLOBECOM, 2021, pp. 1–6.
- [35] B. Zhao, K. Peng, K. Zhang, H. Sun, SerFlow: A multi-stage service-enhanced mechanism for workflow applications in CPSs with end-edge-cloud collaboration, *IEEE Internet Things J.* (2025) <http://dx.doi.org/10.1109/JIOT.2025.3577233>.
- [36] B. Hu, J. Du, J. Zhang, X. Chu, Computation offloading and resource allocation in mixed cloud/vehicular-fog computing systems, *IEEE Trans. Mob. Comput.* (2025) <http://dx.doi.org/10.1109/TMC.2025.3556315>.
- [37] B. Hu, W. Zhang, Y. Yin, J. Du, F.R. Yu, X. Chu, Soft actor-critic-based offloading and resource allocation optimisation for digital twin-aided vehicular networks, *IEEE Trans. Veh. Technol.* (2026) <http://dx.doi.org/10.1109/TVT.2026.3656450>.
- [38] K. Peng, B. Zhao, M. Bilal, X. Xu, Reliability-aware computation offloading for delay-sensitive applications in MEC-enabled aerial computing, *IEEE Trans. Green Commun. Netw.* 6 (3) (2022) 1511–1519, <http://dx.doi.org/10.1109/TGCN.2022.3162584>.
- [39] Y. Cheng, J. Ma, Z. Liu, Z. Li, Y. Wu, C. Dong, R. Li, A privacy-preserving and reputation-based truth discovery framework in mobile crowdsensing, *IEEE Trans. Dependable Secur. Comput.* 20 (6) (2023) 5293–5311.
- [40] C. Li, C. Liu, P. Liu, X. Li, W. Qiu, L. Lei, Y. Jin, Blockchain-based privacy-preserving and accountable mobile edge outsourcing computing framework for the metaverse, *IEEE Trans. Green Commun. Netw.* 9 (2) (2025) 711–724.
- [41] F. Tian, An agri-food supply chain traceability system for China based on rfid & blockchain technology, in: 2016 13th International Conference on Service Systems and Service Management, ICSSSM, 2016, pp. 1–6.
- [42] Y. Liu, F.R. Yu, X. Li, H. Ji, V.C.M. Leung, Blockchain and machine learning for communications and networking systems, *IEEE Commun. Surv. Tutor.* 22 (2) (2020) 1392–1431.
- [43] Z. Xiong, Y. Zhang, D. Niyato, P. Wang, Z. Han, When mobile blockchain meets edge computing, *IEEE Commun. Mag.* 56 (8) (2018) 33–39.
- [44] M.M. Salim, J.H. Park, Federated learning-based secure electronic health record sharing scheme in medical informatics, *IEEE J. Biomed. Health Inform.* 27 (2) (2023) 617–624.
- [45] M. Li, F.R. Yu, P. Si, W. Wu, Y. Zhang, Resource optimization for delay-tolerant data in blockchain-enabled IoT with edge computing: A deep reinforcement learning approach, *IEEE Internet Things J.* 7 (10) (2020) 9399–9412.
- [46] A. Lakhan, M.A. Mohammed, S. Kozlov, J.J. Rodrigues, Mobile-fog-cloud assisted deep reinforcement learning and blockchain-enable IoMT system for healthcare workflows, *Trans. Emerg. Telecommun. Technol.* 35 (4) (2024) e4363.
- [47] J. Zhu, X. Wang, H. Huang, S. Cheng, M. Wu, A NSGA-II algorithm for task scheduling in UAV-enabled MEC system, *IEEE Trans. Intell. Transp. Syst.* 23 (7) (2022) 9414–9429.
- [48] Z. Hu, B. Liu, A. Shen, J. Luo, Blockchain-based resource allocation mechanism for the internet of vehicles: Balancing efficiency and security, *IEEE Trans. Netw. Serv. Manag.* 21 (4) (2024) 3971–3987.
- [49] F. Zhao, Y. Chen, Y. Zhang, Z. Liu, X. Chen, Dynamic offloading and resource scheduling for mobile-edge computing with energy harvesting devices, *IEEE Trans. Netw. Serv. Manag.* 18 (2) (2021) 2154–2165.
- [50] X. Yang, X. Yang, X. Yi, I. Khalil, X. Zhou, D. He, X. Huang, S. Nepal, Blockchain-based secure and lightweight authentication for internet of things, *IEEE Internet Things J.* 9 (5) (2022) 3321–3332.
- [51] B. Mao, Y. Liu, Z. Wei, H. Guo, Y. Xun, J. Wang, J. Liu, N. Kato, A blockchain-enabled cold start aggregation scheme for federated reinforcement learning-based task offloading in zero trust LEO satellite networks, *IEEE J. Sel. Areas Commun.* 43 (6) (2025) 2172–2182.
- [52] D. Larimer, Delegated proof-of-stake (DPOS). Bitshare whitepaper (2014), 2014.
- [53] P. Belotti, C. Kirches, S. Leyffer, J. Linderoth, J. Luedtke, A. Mahajan, Mixed-integer nonlinear optimization, *Acta Numer.* 22 (2013) 1–131.
- [54] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, *IEEE Trans. Evol. Comput.* 6 (2) (2002) 182–197.